



UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO



DCC301– ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES – 2026.2

PROF. DR. HERBERT OLIVEIRA ROCHA

ALINE RIBEIRO LACERDA

ANA BEATRIZ SILVA MELO

[Link do Repositório](#)

LABORATÓRIO DE CIRCUITOS

Subsistema de Memória e Unidade de Controle

BOA VISTA, RR

2026

ALINE RIBEIRO LACERDA

ANA BEATRIZ SILVA MELO

LABORATÓRIO DE CIRCUITOS

Subsistema de Memória e Unidade de Controle

Relatório do Laboratório de Circuitos,
Projeto integrador sobre Subsistema
de Memória e Unidade de Controle da
disciplina de Arquitetura e
Organização de Computadores.

Docente: Prof. Herbert Oliveira Rocha

BOA VISTA, RR

2026

SUMÁRIO

1. INTRODUÇÃO.....	5
2.PARTE I - SUBSISTEMA DE MEMÓRIAS.....	5
2.1 Mapa de memória.....	5
2.1.1 Decodificação da tabela CS.....	6
2.1.2 T-01 Seleção de região.....	7
2.2 Módulos de memória e seleção de saída.....	9
2.3 Bit de paridade Ímpar.....	10
2.3.1 Blocos de paridade.....	10
2.3.2 Expressão booleana.....	11
2.3.3 T-02 Erro de paridade.....	12
2.5 Cache.....	13
2.5.1 T-03 Teste de Falta Compulsória.....	14
2.5.2 T-04 Acerto Mesmo Bloco.....	15
2.5.3 T-05 Falta por conflito.....	15
3. PARTE II - UNIDADE DE CONTROLE.....	16
3.1 Detector de sequência “101”	16
3.2 Máquina de estados da ULA.....	18
3.2.1 Tabela de transição da unidade de controle.....	18
4. CONCLUSÃO.....	19
5.DECLARAÇÃO DO USO DE IA.....	19

1. INTRODUÇÃO

Este relatório técnico apresenta o desenvolvimento e a simulação de um módulo de memória de 64 bytes com decodificação de endereços e uma cache de 4 linhas medida sobre um traço de leitura. Toda essa implementação foi realizada no software Logisim versão 5.0.0, simulador de circuitos digitais utilizado para a montagem e simulação do projeto. O objetivo principal deste trabalho é implementar e analisar um subsistema de memória composto por quatro regiões de 16 bytes, ROM, RAM, banco de registradores e região de entrada/saída selecionadas por meio da decodificação dos bits mais significativos do endereço.

2. PARTE I - SUBSISTEMA DE MEMÓRIAS

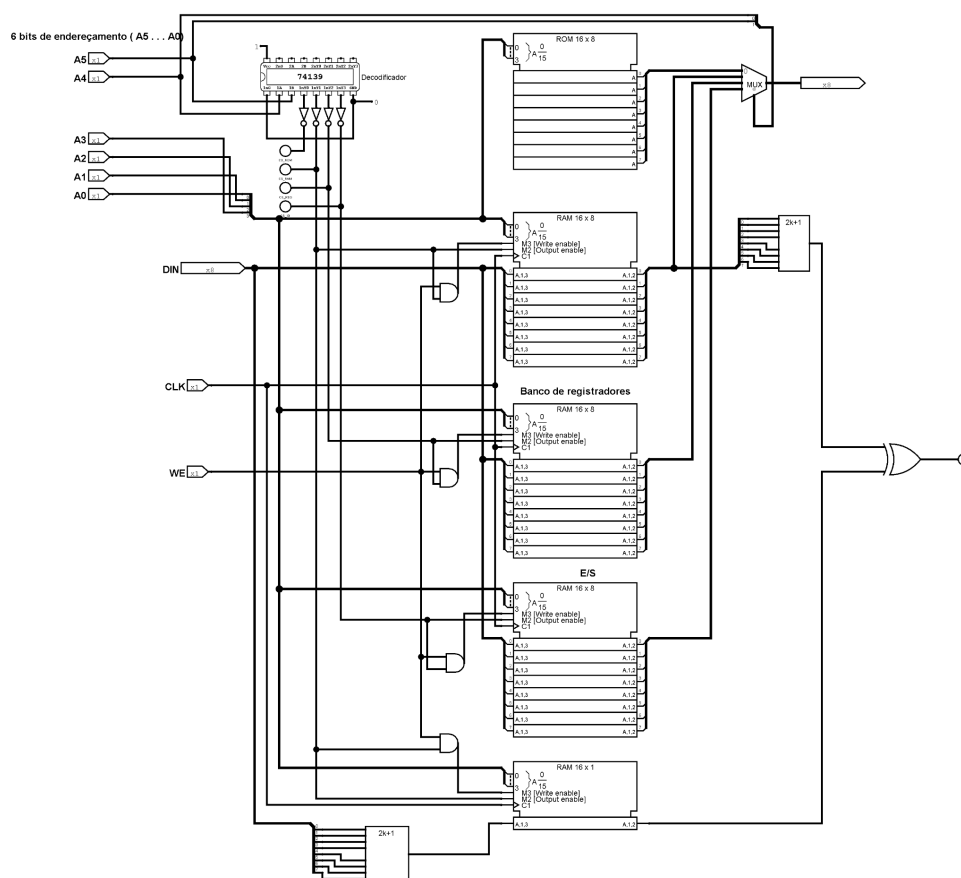
2.1 Mapa de memória

O módulo de memória é um espaço de 64 bytes dividido em 4 regiões de 16 bytes que são usados endereços de 6 bits, de A5 até A0. Os dois bits mais significativos A5 e A4 são usados para identificar as regiões de memória, já os outros 4 bits, A3 até A0, vão selecionar uma das 16 posições dentro da região escolhida. Depois, um multiplexador de 4 entradas é usado para juntar as saídas e selecionar o dado da região escolhida, encaminhando-o para a saída do módulo de memória

Tabela 1

Faixa de endereços	Regiões	tamanho	signal de seleção
0x00-0x0F	ROM	16 bytes	CS_ROM = 00
0x10-0x1F	RAM	16 bytes	CS_RAM = 01
0x20-0x2F	Banco de registradores	16 bytes	CS_REG = 10
0x30-0x3F	Região E/S	16 bytes	CS_IO = 11

Figura 1 - Mapa de memória



2.1.1 Decodificação da tabela CS

O decodificador $2 \rightarrow 4$ representado pelo componente 74139 no logisim, recebe os bits A5 e A4 nas entradas 1B e 1A respectivamente. A partir dessas entradas são gerados sinais de saída nas portas 1nY0, 1nY1, 1nY2 e 1nY3, correspondente, respectivamente aos sinais CS_ROM, CS_RAM, CS_REG, CS_IO. Como o 74139 possui saídas ativas em nível lógico baixo, as saídas do componente são invertidas por meio de portas NOT, de modo que os sinais de seleção utilizados no circuito apresentem nível lógico 1 quando a respectiva região da memória estiver selecionada.

Figura 2 - 74139: Dual 2-line to 4-line decoder

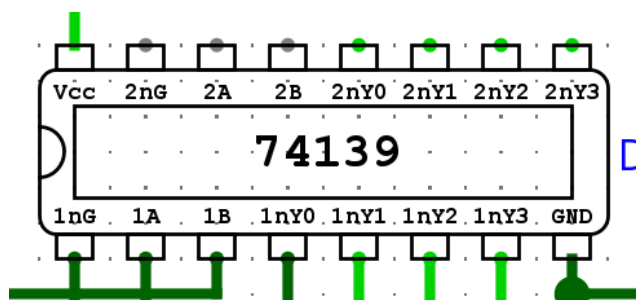


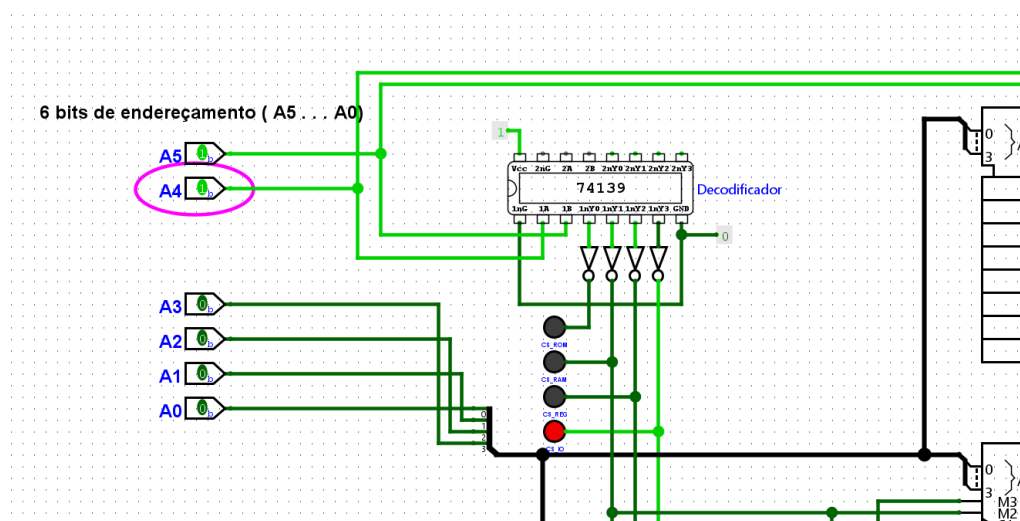
Tabela 2

Tabela verdade de A5 e A4 dos sinais de CS					
A5	A4	CS_ROM	CS_RAM	CS_REG	CS_IO
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

2.1.2 T-01 Seleção de região

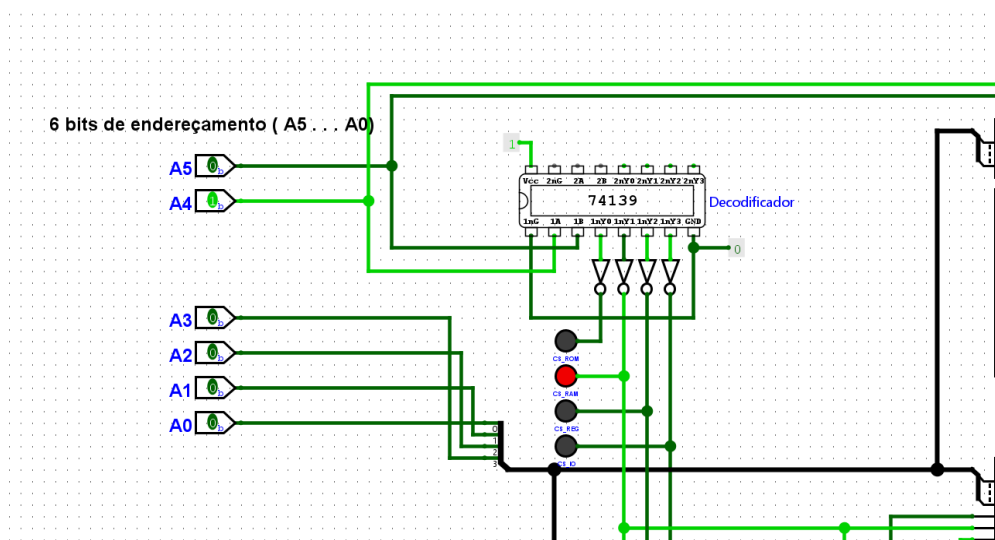
No circuito o pino A5 está conectado à porta de entrada 1B do componente 74139 e o pino A4 está conectado à porta de entrada 1A.

Figura 3 - Teste com A5 e A4 = 00



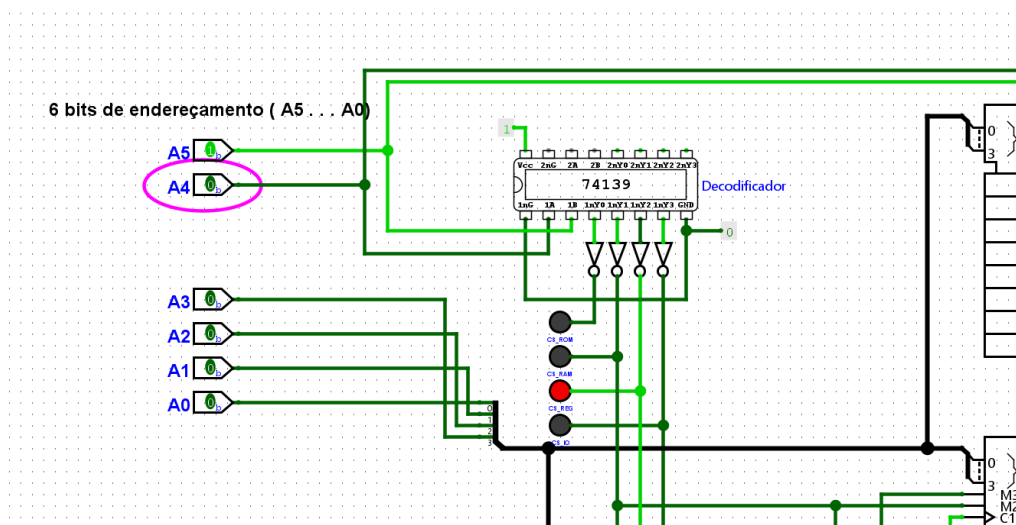
A região correspondente ao código 00 é a ROM, e o sinal que sai do pino 1nY0 é invertido com uma porta lógica NOT para assumir o nível lógico alto (1).

Figura 4 - Teste com A5 e A4 = 01



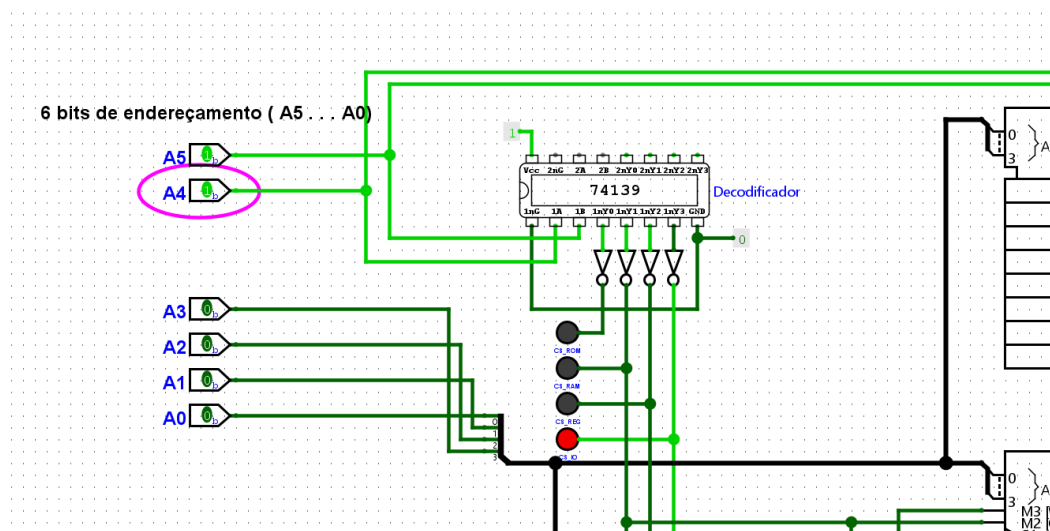
A região correspondente ao código 01 é a RAM, e o sinal que sai do pino 1nY1 é invertido com uma porta lógica NOT para assumir o nível lógico alto (1).

Figura 5 - Teste com A5 e A4 = 10



A região correspondente ao código 10 é o Banco de Registrador, e o sinal que sai do pino 1nY2 é invertido com uma porta lógica NOT para assumir o nível lógico alto (1).

Figura 6 - Teste com A5 e A4 = 11



A região correspondente ao código 11 é o E/S, e o sinal que sai do pino 1nY3 é invertido com uma porta lógica NOT para assumir o nível lógico alto (1).

2.2 Módulos de memória e seleção de saída

O espaço de memória, de 64 bytes, foi dividido em quatro regiões de 16 bytes, sendo elas ROM, RAM, Banco de registradores e E/S, todos utilizam dados de 8 bits. Para apontar as posições internas de cada módulo são utilizados os bits A3 até o A0, enquanto o sinal de CS indica a região.

M0: ROM

- Na entrada de endereço recebe A3, A2, A1 e A0.
- Sua saída de dados é conectada à primeira entrada (entrada 0) do multiplexador.

M1: RAM

- Na entrada de endereço recebe A3, A2, A1 e A0.
- Sua saída é conectada à segunda entrada (entrada 1) do multiplexador.

M2: Banco de registradores

- A entrada de endereço com largura de 4 bits recebe A3, A2, A1 e A0.
- Sua saída de dados é conectada à terceira entrada (entrada 2) do multiplexador.

M3: E/S

- Na entrada de endereço recebe A3, A2, A1 e A0.
- Sua saída de dados na quarta entrada do multiplexador.

MUX

O multiplexador 4→1, com largura de dados de 8 bits, é responsável por selecionar qual das quatro saídas será encaminhada para a saída do módulo de memória. Para isso, os bits A5 e A4 são utilizados como sinais de seleção do MUX, mantendo a correspondência entre as quatro combinações de endereço e as regiões de memória. Dessa forma, para cada endereço, apenas o dado correspondente à região selecionada é encaminhado para a saída.

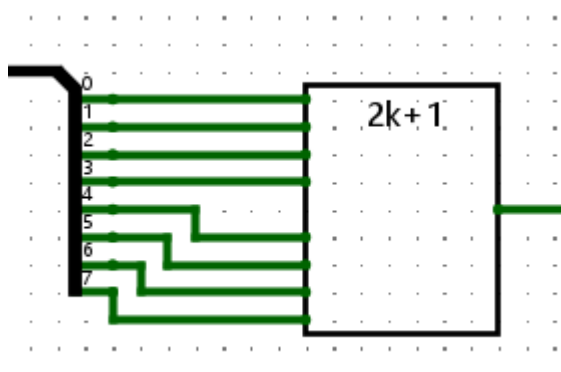
2.3 Bit de paridade Ímpar

O bit de paridade é um método simples para detectar erros de leitura ou escrita. A paridade ímpar adiciona o bit extra (0 ou 1) no final do conjunto de um dado binário para garantir que o total de bits “1” seja ímpar.

Caso o dado tenha uma quantidade par de números “1” o bit de paridade gerado é 1, entretanto se a quantidade de “1” for ímpar o bit gerado será 0.

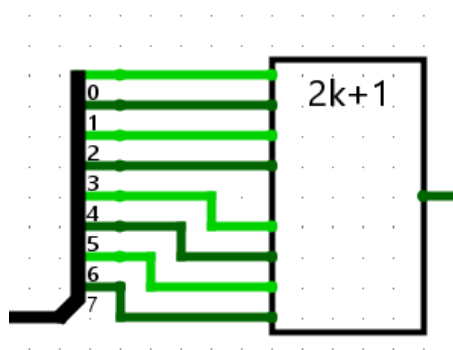
2.3.1 Blocos de paridade

Figura 7 - Paridade ímpar para escrita



O componente de paridade ímpar para a escrita recebe os 8 bits de dados do pino de entrada DIN por meio de um distribuidor e gera de saída um bit (1 ou 0) que será gravado em uma RAM 16 x 1.

Figura 8 - Paridade ímpar para leitura



O componente de paridade ímpar para a leitura recebe os 8 bits de dados vindos do barramento de saída dos módulos de memória por meio de um distribuidor e gera 1 bit de paridade recalculado. Este bit é enviado para uma das entradas de uma porta XOR, onde é comparado com o bit de paridade armazenado anteriormente na RAM 16 x 1 para validar a integridade da leitura.

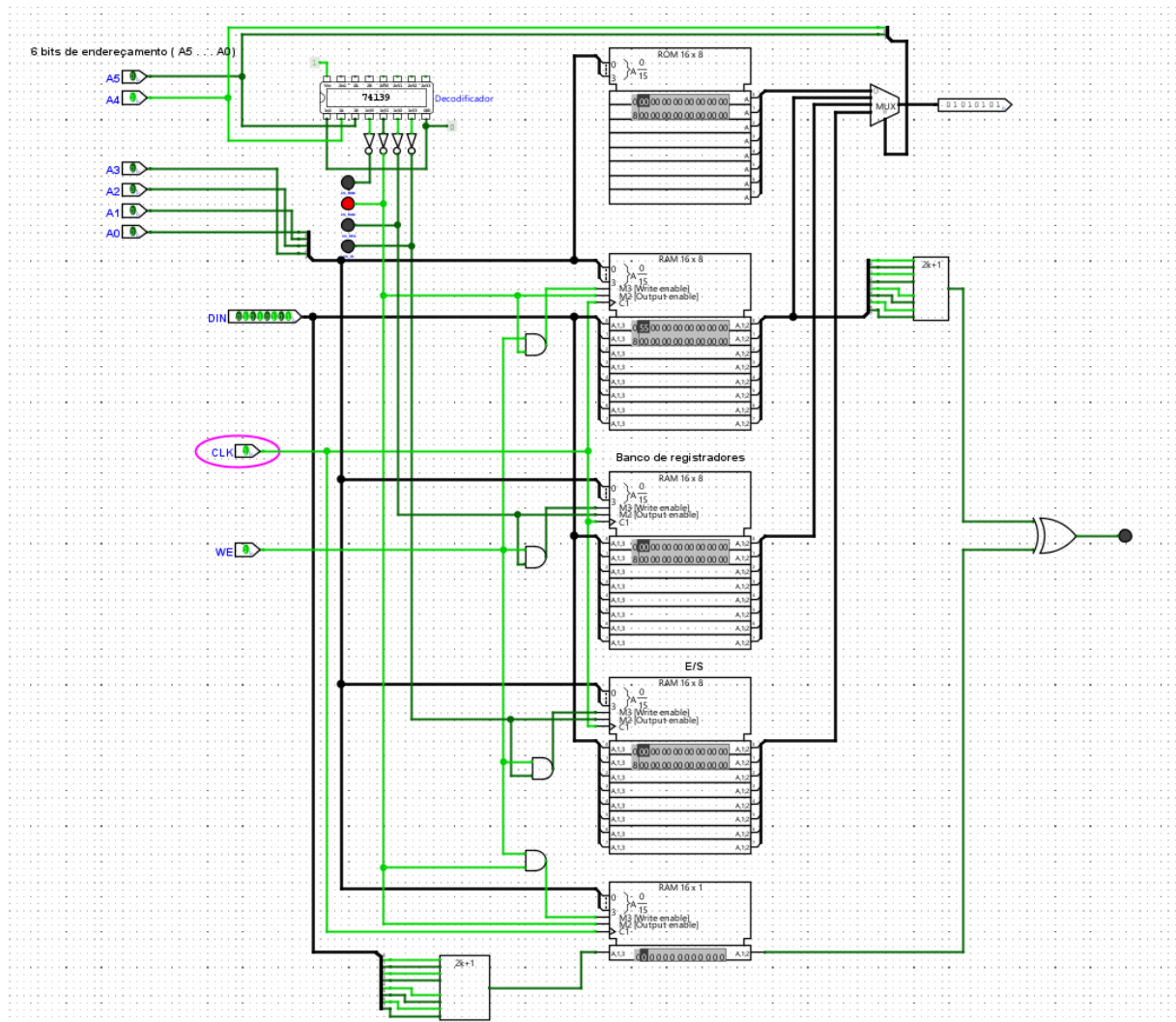
2.3.2 Expressão booleana

A expressão abaixo corresponde a uma paridade ímpar com 8 entradas e 7 portas XOR com uma porta NOT na última saída.

$$P = \overline{D_0 \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7}$$

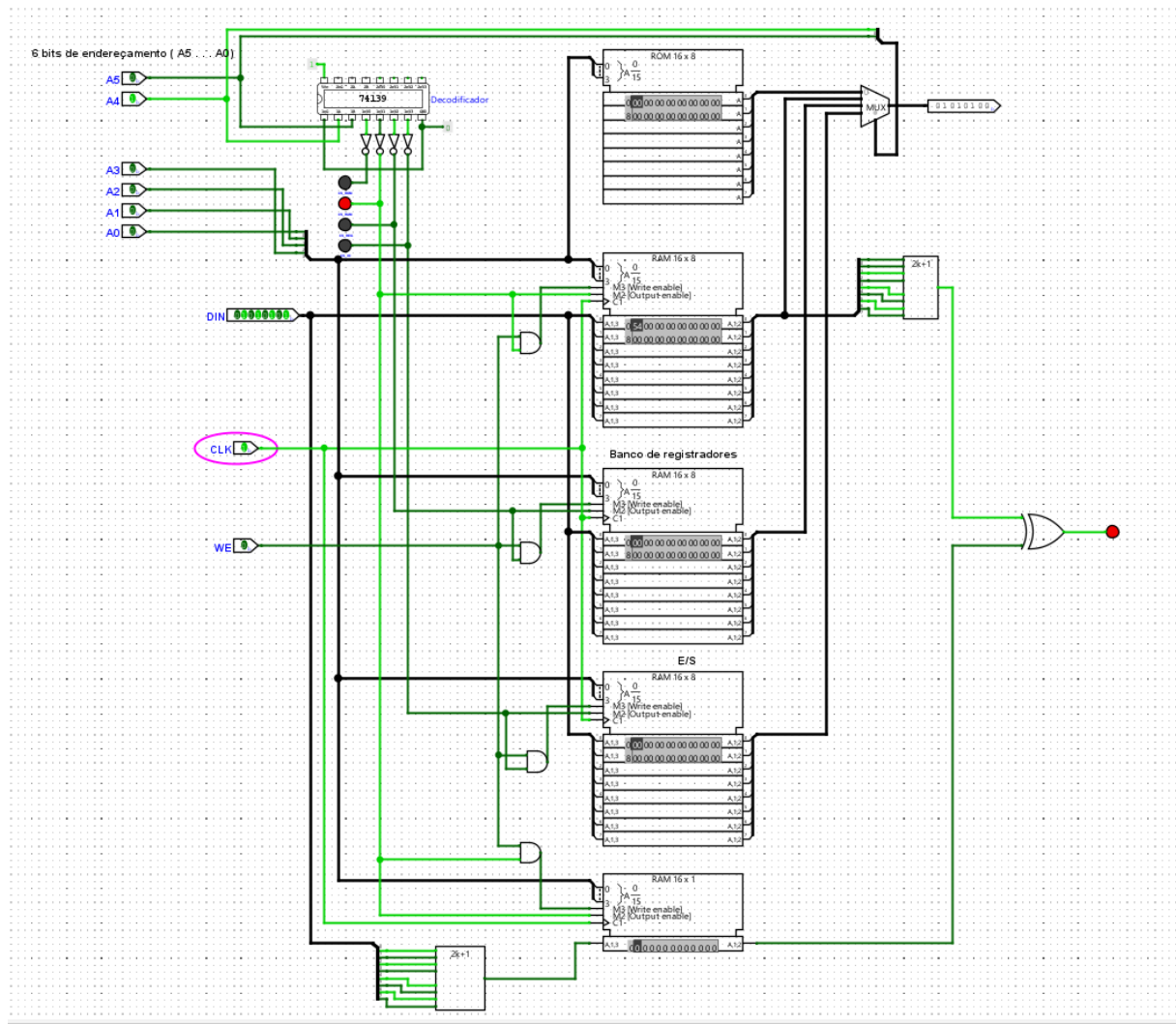
2.3.3 T-02 Erro de paridade

Figura 9 - Teste de gravar 0x55



Na gravação do valor 0x55 (representado em binário por 01010101), o dado foi inserido pelo pino DIN. Com o sinal WE (*Write Enable*) ativo e o clock em nível alto, o dado foi gravado na memória RAM no endereço 0000. Simultaneamente, o bit de paridade correspondente foi gerado e armazenado na memória RAM 16 x 1.

Figura 10 - teste de leitura sinal de erro



Na leitura, o valor do dado foi alterado diretamente para 0x54 (01010100) para simular um erro na transmissão/armazenamento. Dessa forma, o bit de paridade recalculado na leitura e o bit de paridade original armazenado na RAM 16 x 1 são enviados para as entradas de uma porta XOR. Como os valores dos bits de paridade são diferentes (houve discrepância), a saída da porta XOR passa para nível lógico alto (1), acendendo o LED de alerta de erro.

2.5 Cache

Foi implementada uma memória cache de mapeamento direto, composta por 4 linhas, com blocos de 2 bytes, totalizando 8 bits de capacidade de dados. Cada linha possui um bit de validade e um rótulo (TAG) de 3 bits, utilizados para determinar se o bloco armazenado corresponde ao endereço solicitado.

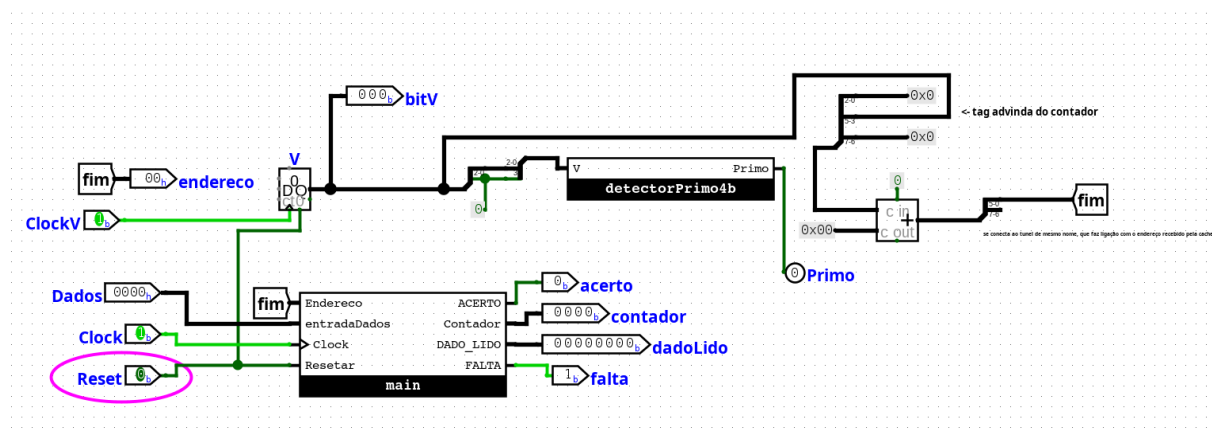
O endereço de memória possui 6 bits e é dividido em três campos: TAG (3 bits), índice (2 bits) e deslocamento (1 bit). O índice determina qual das quatro linhas da cache será consultada, enquanto o deslocamento identifica qual dos dois bytes pertencentes ao bloco deve ser acessado. A TAG é comparada com a TAG armazenada na linha selecionada para verificar se o bloco solicitado está presente.

A detecção de acerto é realizada comparando os três bits da TAG solicitada com os três bits da TAG armazenada. As comparações são feitas por portas XOR, cujas saídas são combinadas por uma porta OR e posteriormente invertidas. O resultado é então combinado com o bit de validade da linha por meio de uma porta AND. Dessa forma, um acesso é considerado acerto (HIT) somente quando a TAG é igual à armazenada e o bit de validade está ativo. Caso contrário, ocorre uma falta (MISS), sendo necessário atualizar a linha correspondente da cache.

A seleção da linha é realizada pelos bits de índice, enquanto o deslocamento é utilizado apenas para selecionar o byte dentro do bloco. Quando ocorre uma falta, o bloco correspondente é carregado na linha indicada pelo índice, juntamente com sua TAG, e o bit de validade dessa linha é ativado. A organização permite observar tanto faltas compulsórias, que ocorrem no primeiro acesso a um bloco ainda não armazenado, quanto faltas por conflito, quando diferentes blocos possuem o mesmo índice e precisam ocupar a mesma linha da cache.

2.5.1 T-03 Teste de Falta Compulsória

Figura 11 - T-03 teste de conflito com detector de primos

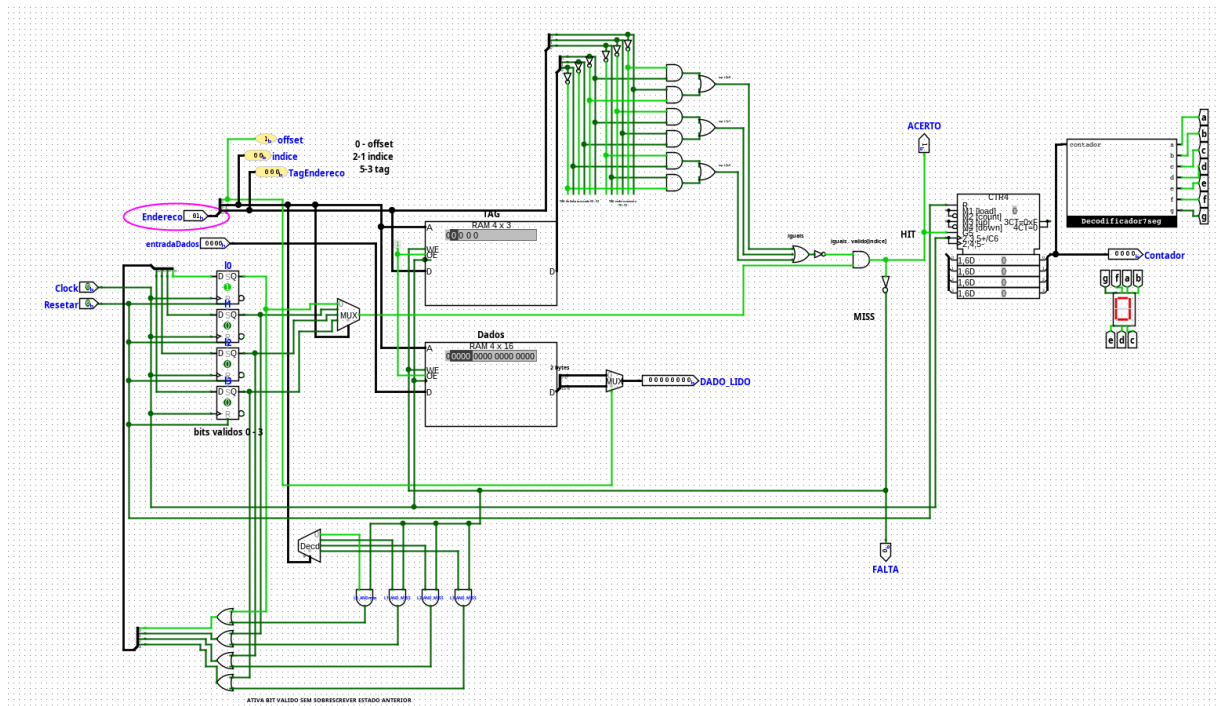


Neste teste, a cache foi inicialmente invalidada, garantindo que nenhuma das quatro linhas possuísse um bloco válido. Em seguida, foi realizado um acesso de leitura ao endereço 0x00. O endereço possui rótulo 000, índice 00 e deslocamento 0. Como a

linha 0 estava inválida antes do acesso, o comparador não produziu um acerto, resultando em ACERTO = 0. O acesso foi classificado como uma falta compulsória, pois o bloco ainda não havia sido carregado na cache. Após a falta, a linha 0 passou a armazenar o rótulo 000 e teve seu bit de validade ativado.

2.5.2 T-04 Acerto Mesmo Bloco

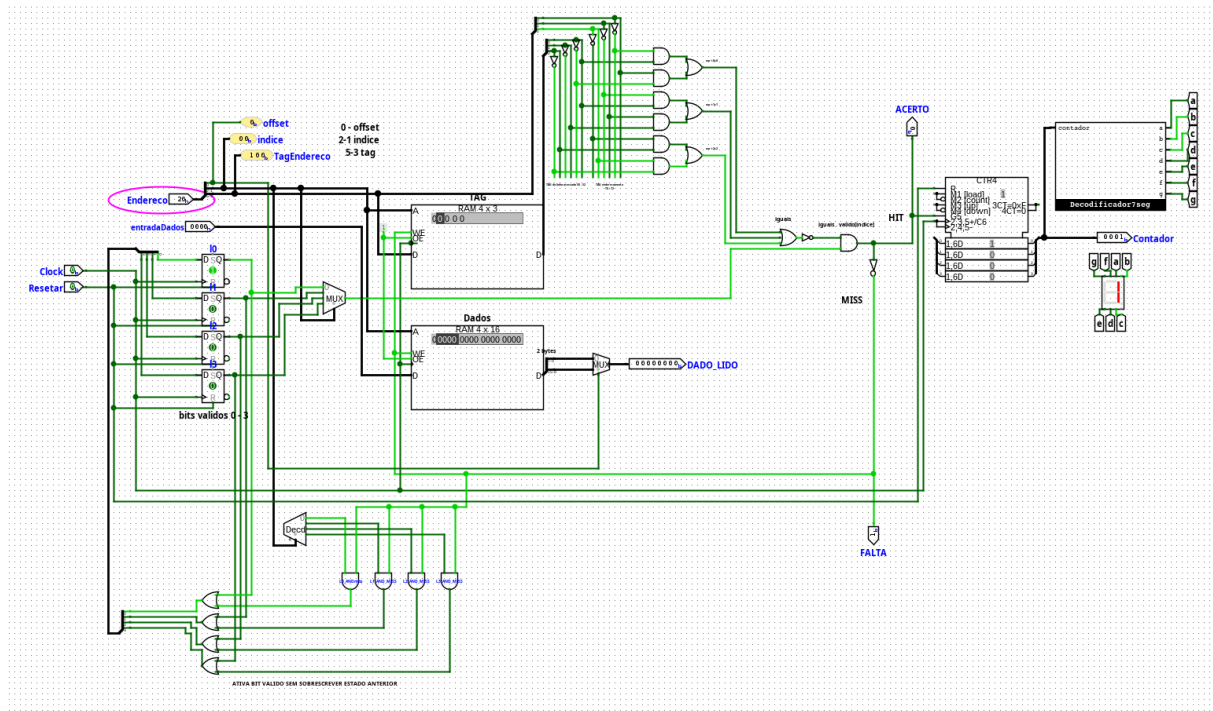
Figura 12 - T-04 Acerto no mesmo bloco



Após a execução do teste T-03, foi realizado um novo acesso de leitura, desta vez ao endereço 0x01. Esse endereço pertence ao mesmo bloco de 2 bytes que o endereço 0x00, possuindo o mesmo rótulo 000 e o mesmo índice 00, mas deslocamento 1. Como o bloco já havia sido carregado na linha 0 durante o teste anterior e seu bit de validade estava ativo, a comparação entre os rótulos apresentou igualdade. Dessa forma, o circuito produziu ACERTO = 1, demonstrando o funcionamento do acesso a outro byte pertencente ao mesmo bloco armazenado na cache.

2.5.3 T-05 Falta por conflito

Figura 13 - T-05 Falta por conflito



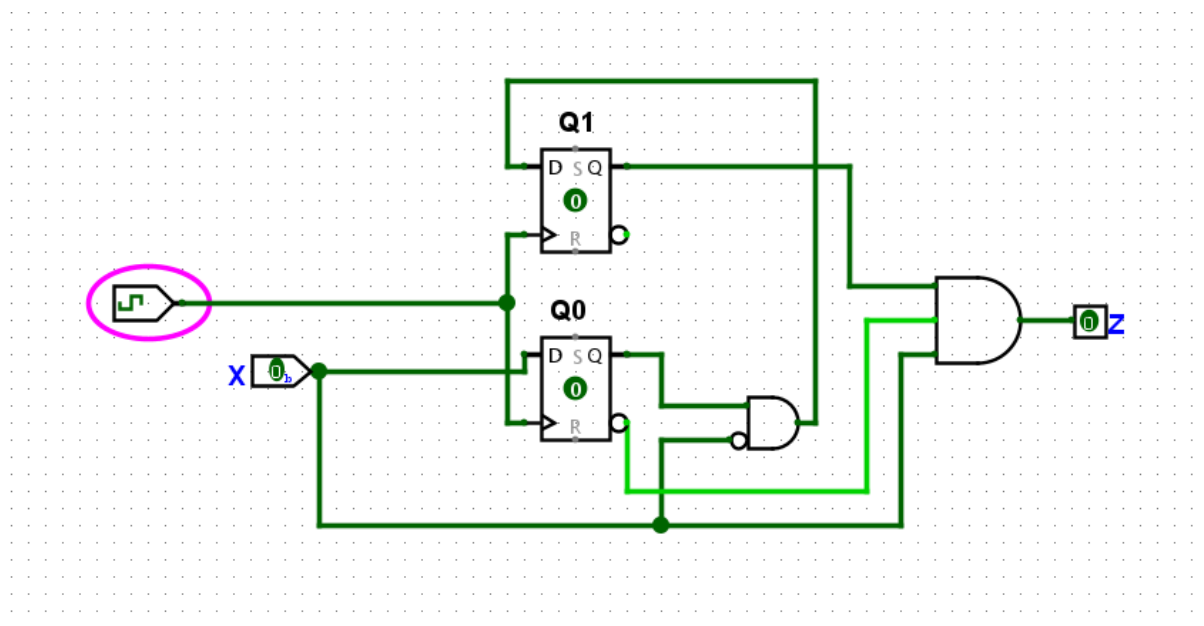
Para este teste, mantendo o estado da cache após T-03, foi realizado um acesso de leitura ao endereço 0x20. Esse endereço possui rótulo 100, índice 00 e deslocamento 0. Portanto, ele é direcionado para a mesma linha 0 utilizada pelo endereço 0x00, mas apresenta um rótulo diferente do que estava armazenado (000). Como consequência, a comparação dos rótulos não produziu igualdade e o circuito apresentou ACERTO = 0. O acesso caracteriza uma falta por conflito, e a linha 0 foi atualizada, passando a armazenar o rótulo 100. Esse comportamento demonstra a característica do mapeamento direto, no qual diferentes blocos podem ser direcionados para a mesma linha da cache.

3. PARTE II - UNIDADE DE CONTROLE

3.1 Detector de sequência “101”

Exercício preparatório de máquina de estados.

Figura 14 - Detetector de sequência 101



O circuito da imagem é uma máquina de mealy síncrona com 3 estados, a saída Z dependo do estado atual de Q1, Q0 e da entrada X

Tabela 3 - Tabela de transição

Estado	Significado
S0 = 00	Nenhum prefixo da sequência visto
S1 = 01	“Viu 1”
S2 = 10	“Viu 10 ”

Tabela 4 - Tabela verdade de transições de estados

Q1	Q0	X	Q1'	Q0'	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	X	X	X

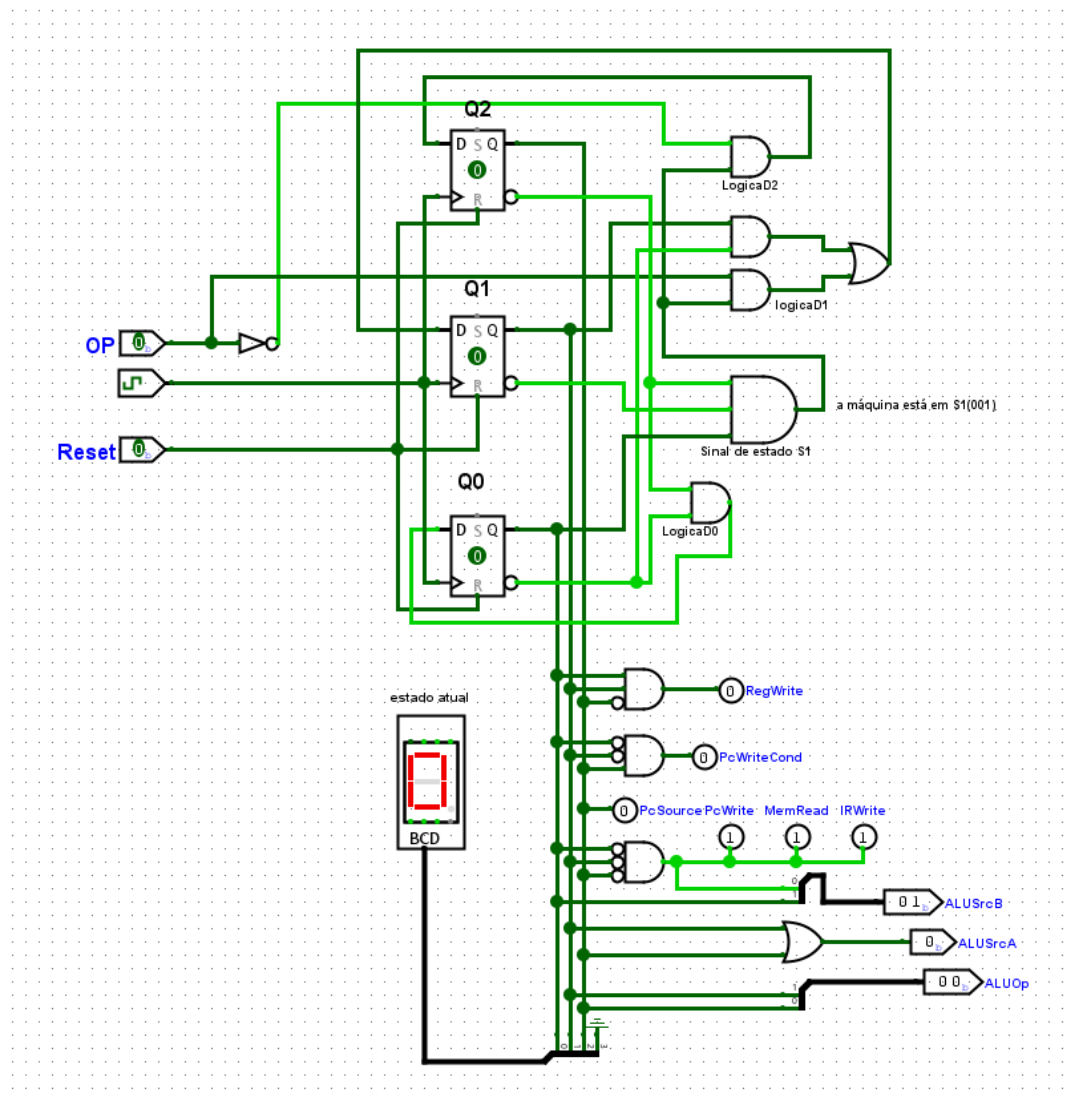
1	1	1	X	X	X
---	---	---	---	---	---

- Q1 e Q0 é o estado atual
- X é a entrada
- Q1' e Q0' é o próximo estado
- Z é a saída

3.2 UC cabeada

Contagem do número de portas usadas, no total foram 17 portas lógicas usadas, 2 NOT, 10 AND e 5 OR.

Figura 15 - Máquina de estados da UC



3.2.1 Tabela de transição da unidade de controle

Tabela 5 - Tabela de Transição Conceitual da UC

Estado atual	OPcode	Próximo estado	o que acontece
S0	X	S1	$IR \leftarrow \text{Mem}[PC];$ $PC \leftarrow PC + 4$
S1	R	S2	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt];$ ULAOut $\leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
S1	beq	S4	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt];$ ULAOut $\leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
S2	X	S3	ULAOut $\leftarrow A \text{ op } B$
S3	X	S0	$\text{Reg}[rd] \leftarrow \text{ULAOut}$
S4	X	S0	ULA calcula $A - B$; se Zero, $PC \leftarrow \text{ULAOut}$

Tabela 6 - Tabela verdade de transições de estados

Q2	Q1	Q0	estado atual	OP	Q2'	Q1'	Q0'	próximo estado
0	0	0	S0	0	0	0	1	S1
0	0	0	S0	1	0	0	1	S1
0	0	1	S1	0	1	0	0	S4
0	0	1	S1	1	0	1	0	S2
0	1	0	S2	0	0	1	1	S3
0	1	0	S2	1	0	1	1	S3
0	1	1	S3	0	0	0	0	S0
0	1	1	S3	1	0	0	0	S0
1	0	0	S4	0	0	0	0	S0

1	0	0	S4	1	0	0	0	S0
1	0	1		0	X	X	X	
1	0	1		1	X	X	X	
1	1	0		0	X	X	X	
1	1	0		1	X	X	X	
1	1	1		0	X	X	X	
1	1	1		1	X	X	X	

OPcode R-type = 1

OPcode beq = 0

Estados Não Utilizados (101, 110, 111). Como 5 estados precisam de 3 Flip-Flops ($2^3 = 8$ combinações), sobram 3 combinações não utilizadas.

3.2.2 testes da máquina de estados

Figura 16 - Mudança do estado de busca para decodificação

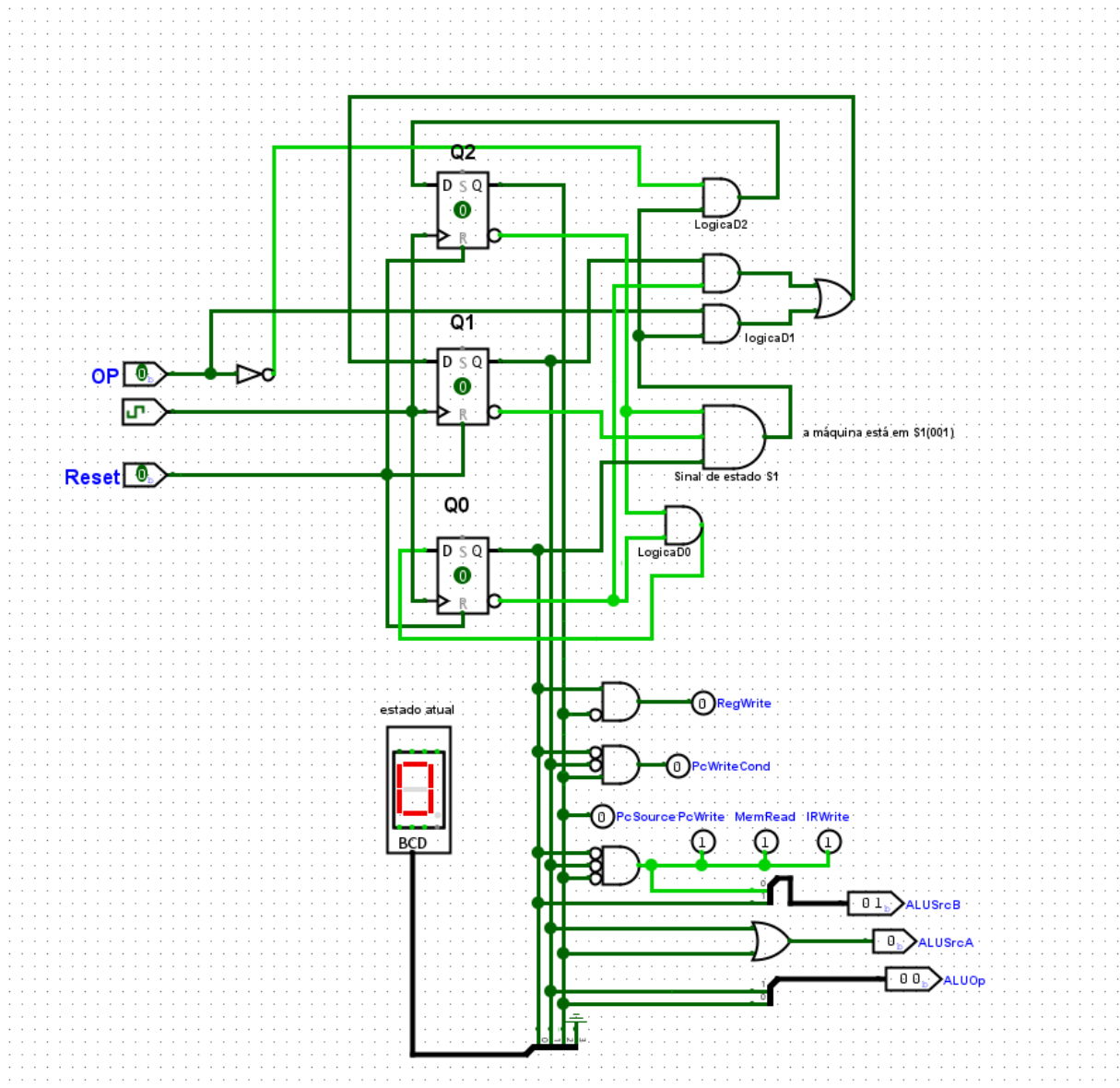
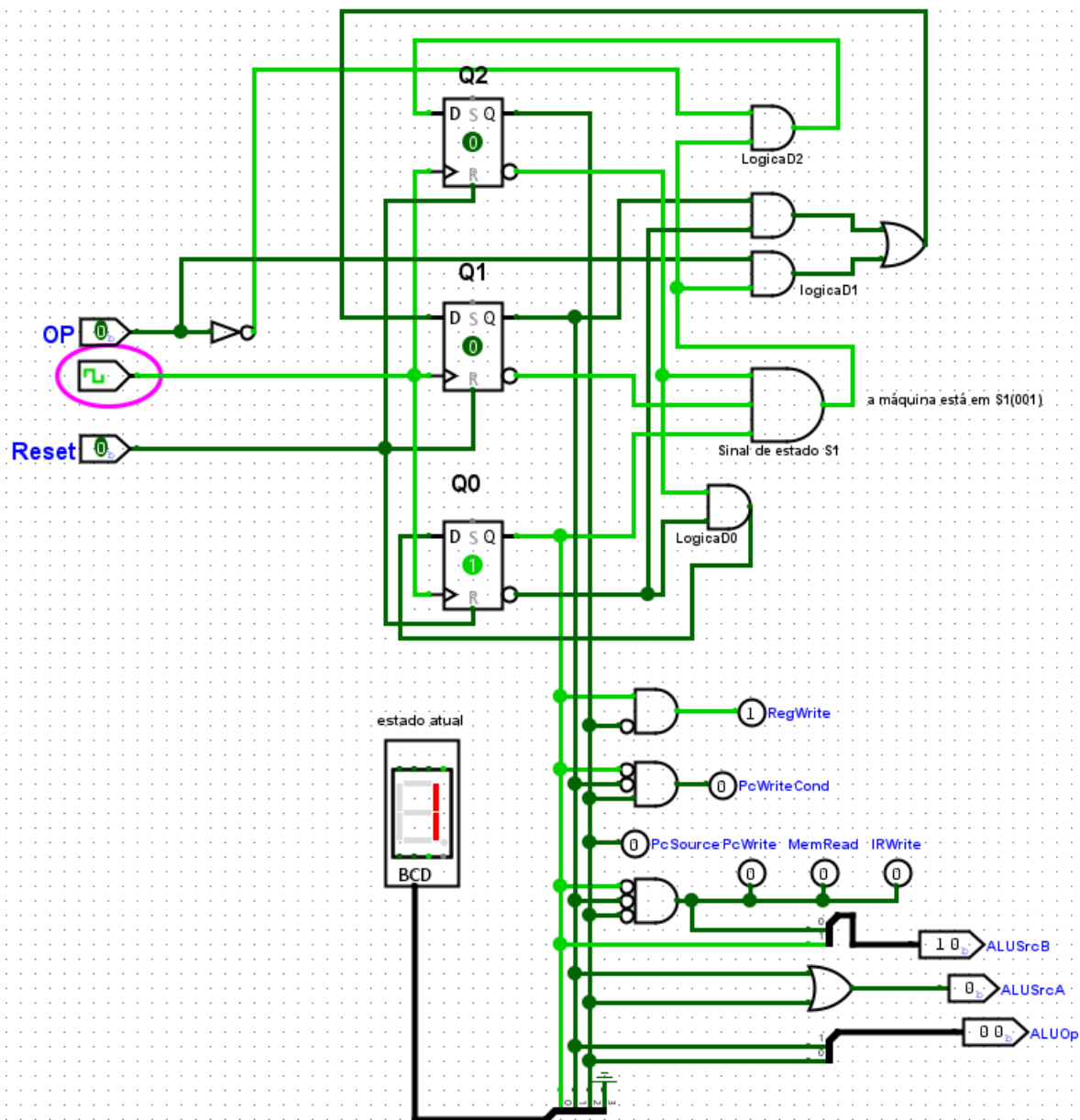


Figura 17 - Mudança do estado de busca para decodificação



Validação da transição do estado inicial de Busca (S_0) para o estado de Decodificação (S_1)

Tabela 7 - Tabela verdade dos sinais de controle

Estado	PcWr	PcWrc	PcSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0	1	0	0	1	1	0	0	01	00

S1	0	0	x	0	x	0	0	10	00
S2	0	0	x	0	x	0	1	00	10
S3	0	0	x	0	x	1	x	xx	xx
S4	0	1	1	0	x	0	1	00	01
não válido	0	0	x	x	0	0	x	xx	xx
não válido	0	0	x	x	0	0	x	xx	xx
não válido	0	0	x	x	0	0	x	xx	xx

Os estados não válidos da tabela foram considerados como *don't cares* na maioria das saídas dos sinais de controle, exceto para PcWr, PcWrC e IRWr, uma vez que foi adotada a opção de forçar o retorno ao estado S0. Deixar a saída desses sinais como *don't care* poderia permitir a escrita no contador de programa ou no registrador de instruções em estados não válidos, podendo causar comportamentos indesejados.

As ocorrências de *don't care* no sinal PcSrc foram implementadas nos estados S1 a S3, devido à ULAOut não ser utilizada para a escrita em PcSrc, já que não há escrita no PC nesses estados. Quanto aos sinais ALUSrcA e ALUSrcB, foi inserido o *don't care* no estado S3, pois será utilizada somente a saída da ULA.

Portanto, os estados de *don't care* nos sinais PcSrc, ALUSrcA, ALUSrcB e ALUOp foram escolhidos por não alterarem registradores importantes, não apresentando risco ao funcionamento do circuito.

3.2.3 Tabela de tempo da UC

Tabela 8 - Tabela de tempo

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0	1	0	0	1	1	0	0	01	00
S1	0	0	x	0	0	0	0	10	00
S2	0	0	x	0	0	0	1	00	10
S3	0	0	x	0	0	1	x	xx	xx
S4	0	1	1	0	0	0	1	00	01

Figura 20 - T-07 estado S0

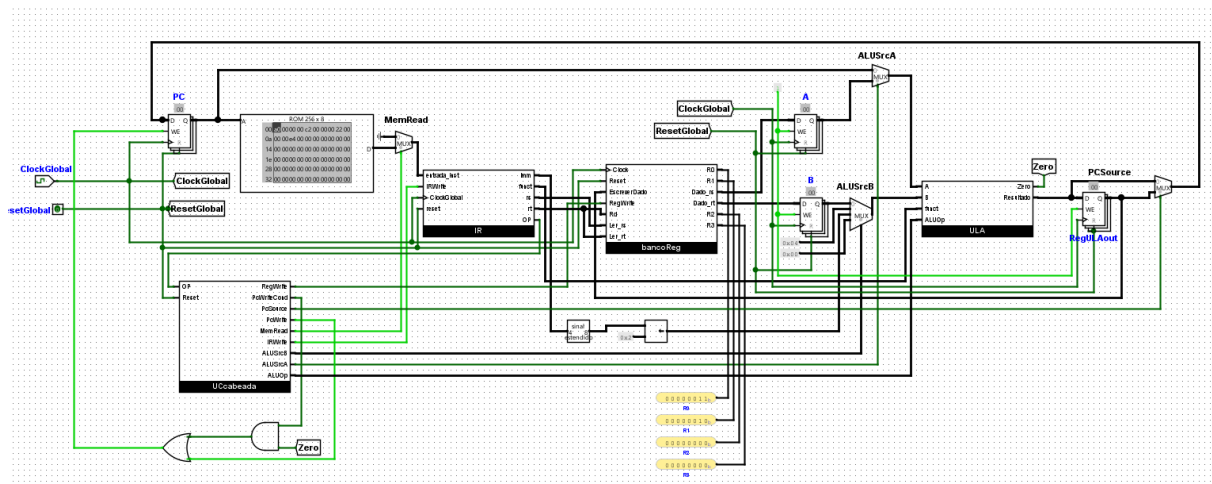


Figura 20 - T-07 estado S1

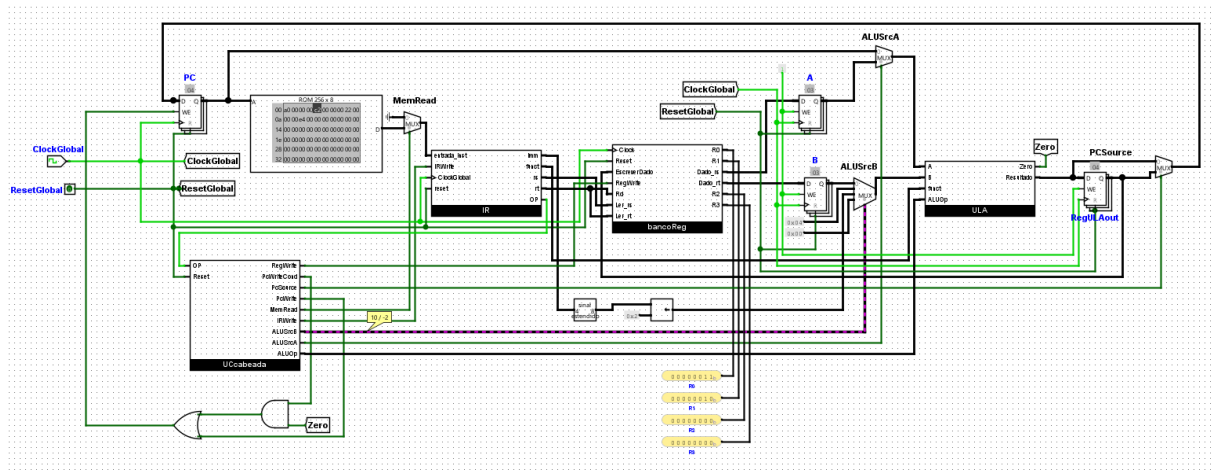
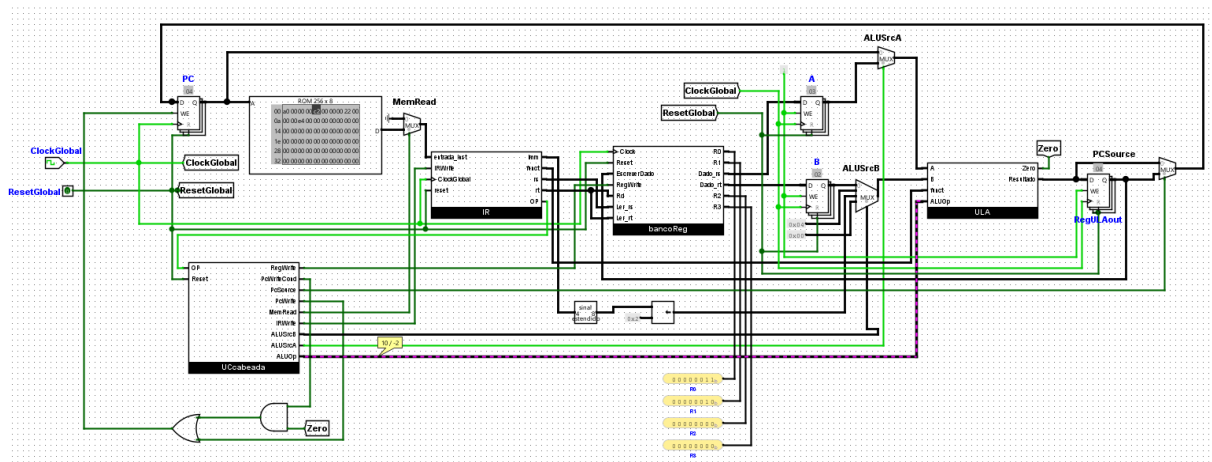


Figura 21- T-07 estado S2



Registadores no estado S2

R0 = 03

R1 = 02

Figura 22- T-07 Display e sinais no estado S2

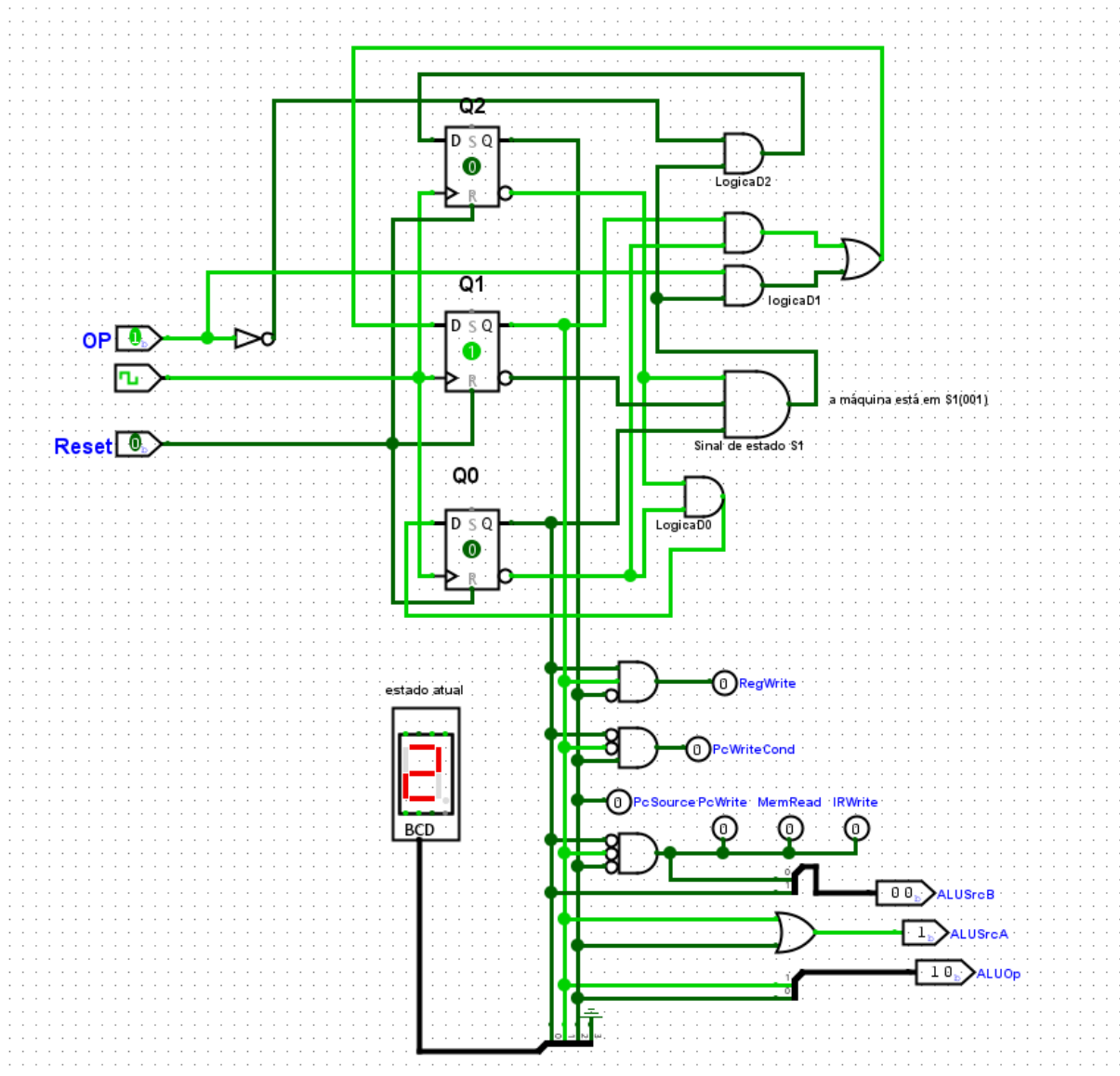


Figura 25- T-08 estado S0

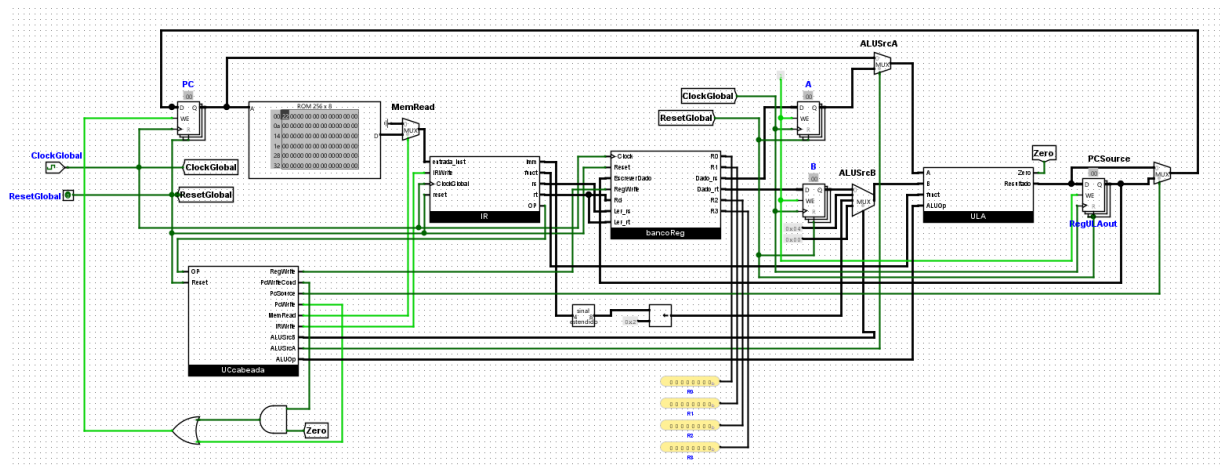


Figura 26- T-08 Display e sinais no estado S0

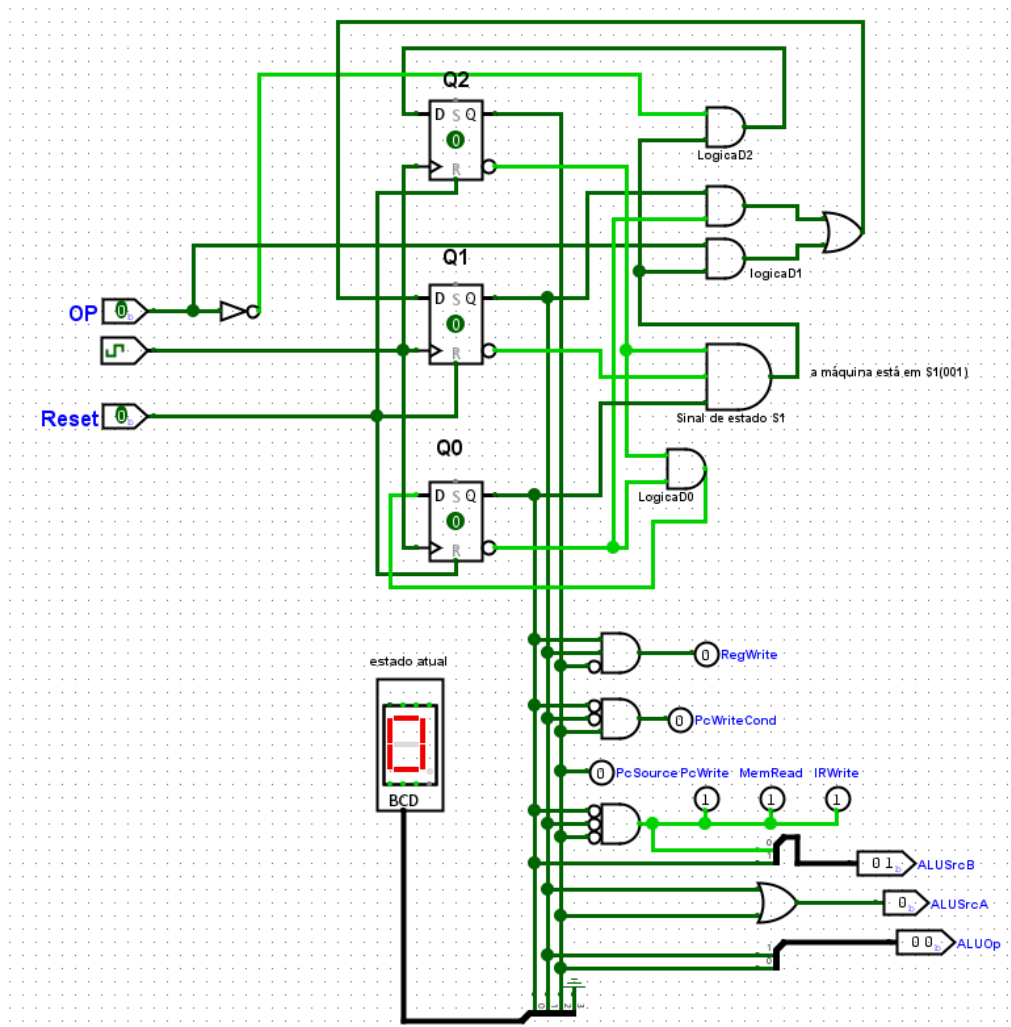
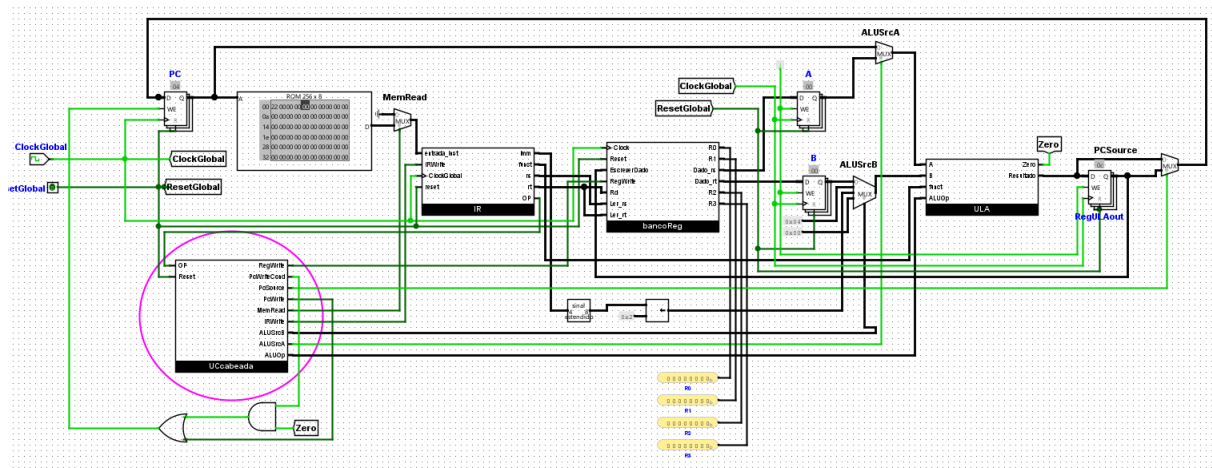


Diagrama de um controlador de máquina de estados finitos (FSM) para um processador de 8 bits. O diagrama mostra três registradores de estado (Q2, Q1, Q0) com entradas D, S, Q e R. As entradas de controle são OP (0), L (1) e Reset (0). A saída de estado S1 indica "a máquina está em S1(001)". A lógica combinacional gera sinais de controle: RegWrite, PcWriteCond, PcSourcePcWrite, MemRead, IRWrite, ALUSrcB, ALUSrcA e ALUOp. Um display BCD mostra o estado atual.

Figura 29- T-08 Estado S4



R0 e R1 são iguais a 0, borda zero = 1

3.2.6.2 Registradores diferentes desvio não tomado

Figura 30- T-08 Estado S1

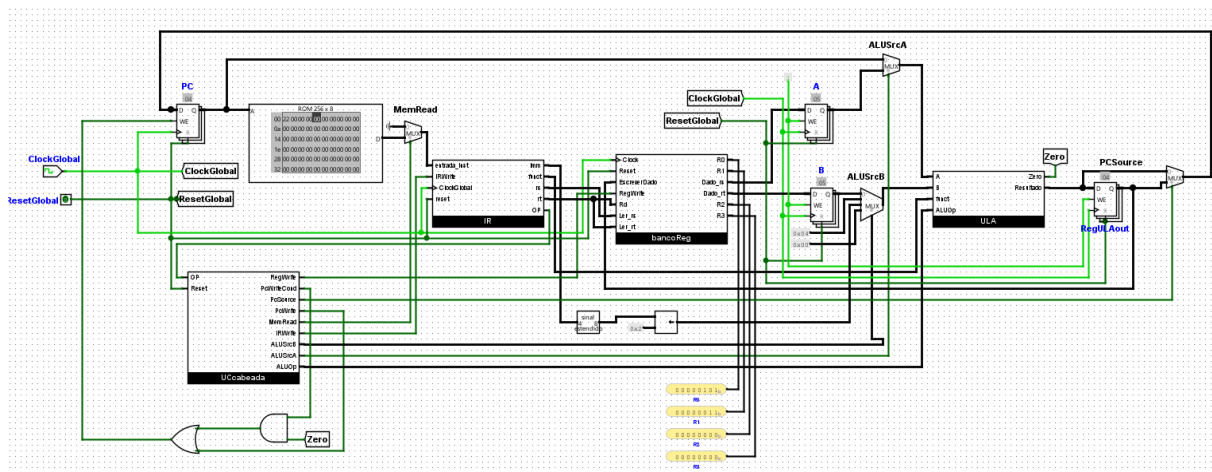


Figura 31- T-08 Display e sinais no estado S1

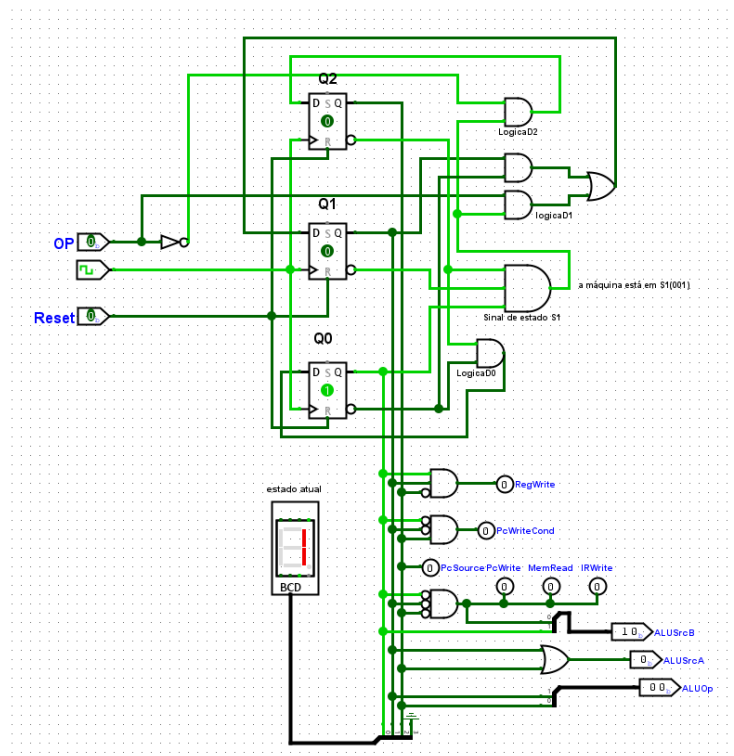
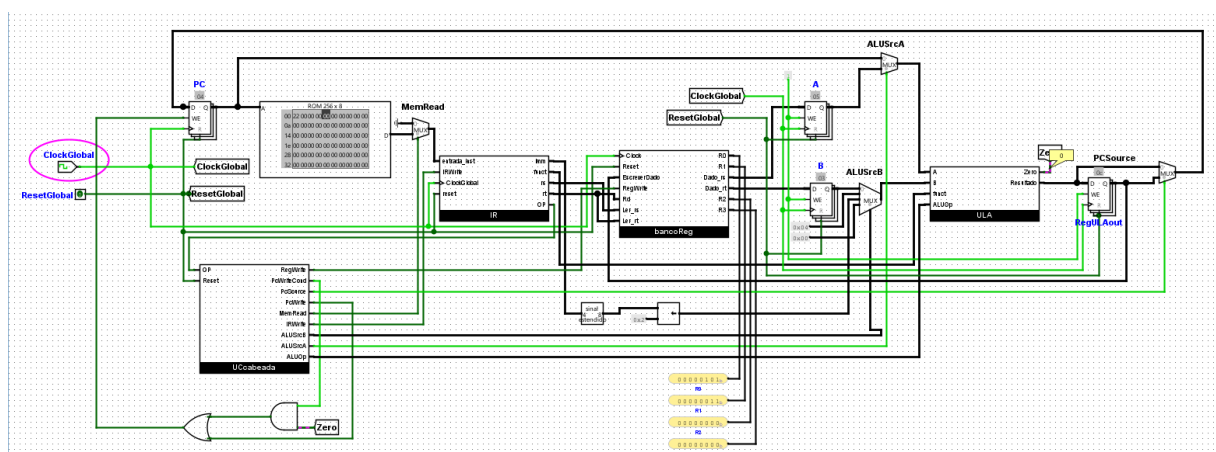


Figura 32- T-08 Estado S4



R0 = 05

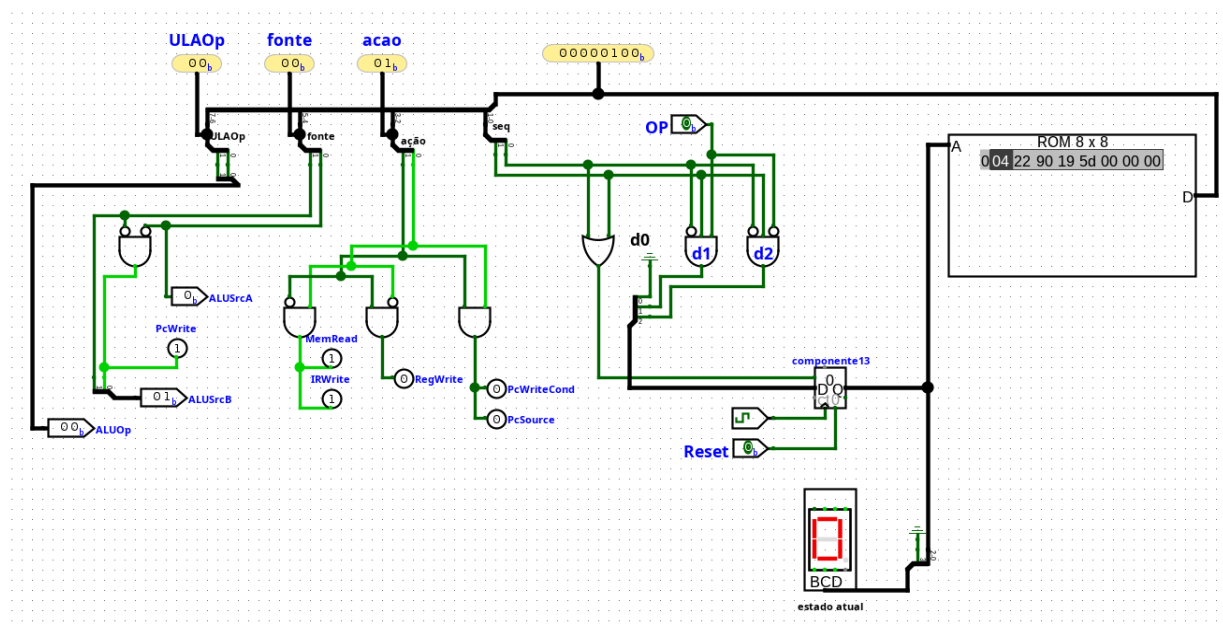
R1 = 03

Saída zero = 0

3.3 UC Microprogramada

A Unidade de Controle Microprogramada aproveita os sinais da Unidade de Controle da Cabeada, tendo a mesma função, entretanto, a geração dos sinais de saída serão feitos pelo microprograma injetado na ROM presente no subcircuito “UCmicroprogramada”.

Figura 20 - Componentes da UC microprogramada



Os pinos de entrada da UC microprogramada são: OP, Clock e Reset. Seus pinos de saída são compostos pelos sinais de controle: ALUOp, ALUSrcA, ALUSrcB, PCWrite, MemRead, IRWrite, RegWrite, PCWriteCond e PCSource.

O microprograma injetado na ROM ocupa 8 posições de 8 bits, onde é dividido em campos de 2 bits: ULAOp, Fonte e Ação. O campo de ULAOp é enviado diretamente para seu respectivo pino de saída, os campos Fonte e Ação foram segmentados para decodificação e decisão de outros sinais de saída.

O conteúdo completo da ROM é gravado no arquivo microcodigo.txt.

Tabela 9 - Tabela do microprograma

Rótulo	Endereço	ULAOp	Fonte	Ação	Seq	Hexadecimal
Busca	0	00	00	01	00	0x04
Decod	1	00	10	00	10	0x22
ExecR	2	10	01	00	00	0x90
ConclR	3	00	01	10	01	0x19
ConclBEQ	4	01	01	11	01	0x5d

Fonte = 00 implica desvio condicional, e PC + 4, por isso Fonte = 00 não pode ser usado em ConclR, pois o PC seria sobrescrito no final de toda instrução R. Desse modo, o valor Fonte = 00 aparece somente em busca quando a escrita no PC é desejada.

4. CONCLUSÃO

5.DECLARAÇÃO DO USO DE IA

Usamos IA para a pesquisa do conceito e funcionamento de componentes, e para a formatação de documentos, além de pesquisas e correções de tabelas e textos.